

# Управление базами данных

Зима-весна 2026

# План лекции

- Повторение пройденного материала
- Введение в проектирование баз данных
  - Нормализация
  - Типы связей между таблицами
  - Примеры
- Элементы SQL в Oracle

# Свойства базы данных

- Самодокументируемость (метаданные как пример)
- Независимость хранилища данных от программ, которые с ними работают
- Целостность данных (возраст родителей не может быть ниже возраста ребенка)
- Целостность транзакций (оплата банковской картой производится только тогда, когда все элементы последовательности выполнены)
- Масштабируемость
- Производительность

# Модели построения баз данных

1. Реляционная – информация организована в виде отношений и связей между ними
2. Нереляционная (NoSQL) - хранят данные без четких связей друг с другом и четкой структуры
  1. Иерархическая модель – четкая упорядоченная структура (дерево). Существует один главный элемент, а остальные считаются подчиненными
  2. Сетевая – иерархическая + связи между элементами (граф)
  3. ...

# Реляционная модель

|           |                 |         |              |        |        |             |
|-----------|-----------------|---------|--------------|--------|--------|-------------|
| Отношение | Целое           | Строка  |              | Целое  |        | Типы данных |
|           | номер           | имя     | должность    | деньги |        | Домены      |
|           | Табельный номер | Имя     | Должность    | Оклад  | Премия | Аттрибуты   |
|           | 2934            | Иванов  | Инженер      | 112    | 40     | Кортежи     |
|           | 2935            | Петров  | Вед. Инженер | 144    | 50     |             |
|           | 2936            | Сидоров | Бухгалтер    | 92     | 35     |             |
| Ключ      |                 |         |              |        |        |             |

# Задачи проектирования баз данных

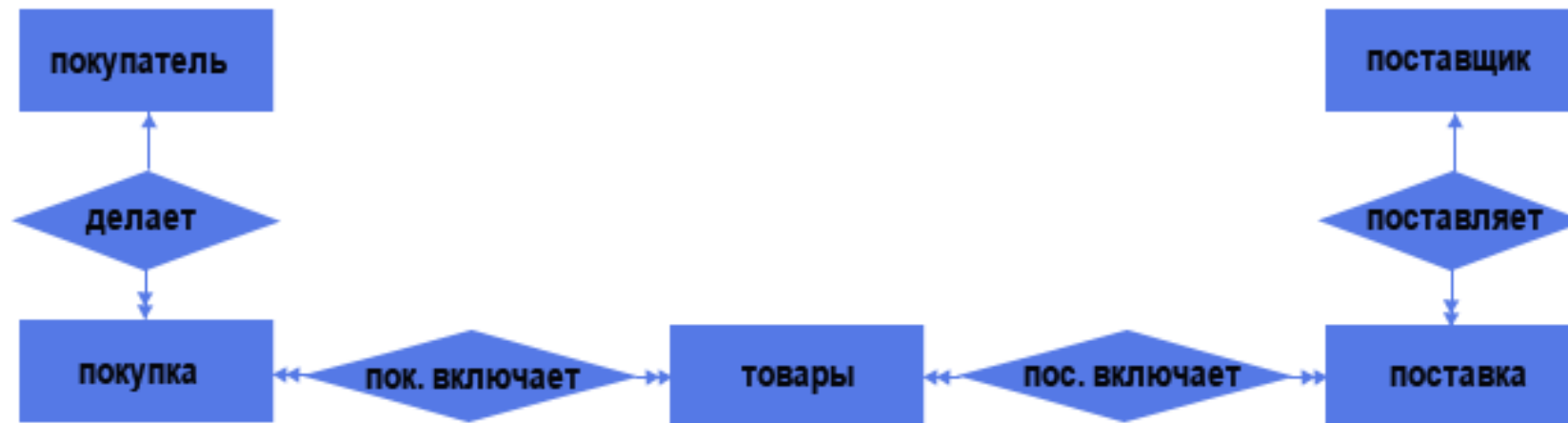
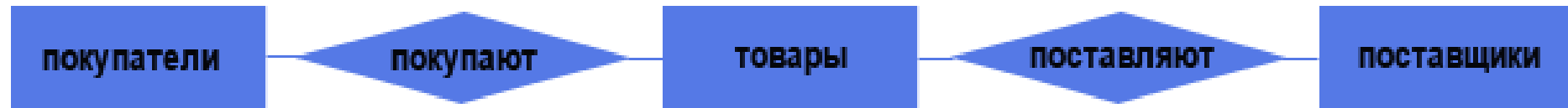
- Обеспечение хранения в БД всей необходимой информации
- Обеспечение возможности получения данных по всем необходимым запросам
- Сокращение избыточности и дублирования данных
- Обеспечение целостности данных

# Этапы проектирования баз данных

- Внешнее представление
  - Совокупность требований со стороны конкретной пользовательской позиции
- Концептуальная схема
  - Совокупность всех требований к данным и сопоставление их друг с другом
- Внутренняя схема
  - Сама база данных

База, в которую уже записаны данные,  
сопротивляется изменениям наиболее дерзко!

# Концептуальное проектирование





# Основные моменты концептуального проектирования баз данных

- Командная работа
- Привлечение наиболее опытных в рассматриваемой предметной области специалистов
- Оценка состояния проектируемой системы не только в краткосрочном, но и в долгосрочном периоде

# Первичный ключ (не NULL!)

Потенциальный ключ – подмножество атрибутов отношения, удовлетворяющее требованиям уникальности и минимальности

Первичный ключ – один из потенциальных ключей отношения, выбранный в качестве основного ключа (уникальный!)

Если потенциальных ключей в отношении несколько, то один из них выбирается в качестве первичного, а оставшиеся называют альтернативными

# Суррогатный ключ

Суррогатный ключ – дополнительное служебное поле, добавленное к уже имеющимся полям таблицы, единственное предназначение которого – служить первичным ключом

- ✓ Неизменность при изменении зависимых полей
- ✓ Гарантированная уникальность
- ✓ Простота модификации
- ✓ Эффективность

# Аномалии в таблицах БД

**Аномалией** называется такая ситуация в таблице БД, которая приводит к противоречию в БД либо существенно усложняет обработку БД. Причиной является излишнее дублирование данных в таблице, которое вызывается наличием функциональных зависимостей от не ключевых атрибутов.

Аномалии-модификации проявляются в том, что изменение одних данных может повлечь просмотр всей таблицы и соответствующее изменение некоторых записей таблицы.

Аномалии-удаления — при удалении какого либо кортежа из таблицы может пропасть информация, которая не связана на прямую с удаляемой записью.

Аномалии-добавления возникают, когда информацию в таблицу нельзя поместить, пока она не полная, либо вставка записи требует дополнительного просмотра таблицы.

# Как быть не должно

| Блюдо | Вид     | Дата       | Продукт | Калорийность | Вес (г) | Поставщик | Город | Страна  | Цена (\$) |
|-------|---------|------------|---------|--------------|---------|-----------|-------|---------|-----------|
| Лобио | Закуска | 01.09.2012 | Фасоль  | 307          | 200     | "Хуанхэ"  | Пекин | Китай   | 0.37      |
| Лобио | Закуска | 01.09.2012 | Лук     | 45           | 40      | "Наталка" | Киев  | Украина | 0.52      |
| Лобио | Закуска | 01.09.2012 | Масло   | 742          | 30      | "Лайма"   | Рига  | Латвия  | 1.55      |
| Лобио | Закуска | 01.09.2012 | Зелень  | 18           | 10      | "Даугава" | Рига  | Латвия  | 0.99      |
| Борщ  | Суп     | 01.09.2012 | Мясо    | 166          | 80      | "Наталка" | Киев  | Украина | 2.18      |
| Борщ  | Суп     | 01.09.2012 | Лук     | 45           | 30      | "Наталка" | Киев  | Украина | 0.52      |
| Борщ  | Суп     | 01.09.2012 | Томаты  | 24           | 40      | "Полесье" | Киев  | Украина | 0.45      |
| Борщ  | Суп     | 01.09.2012 | Рис     | 334          | 50      | "Хуанхэ"  | Пекин | Китай   | 0.44      |
| Борщ  | Суп     | 01.09.2012 | Масло   | 742          | 15      | "Полесье" | Киев  | Украина | 1.62      |
| Борщ  | Суп     | 01.09.2012 | Зелень  | 18           | 15      | "Наталка" | Киев  | Украина | 0.88      |

# Примеры аномалий

| ID Сотрудника | ФИО                      | Должность   | ID Отдела | Название отдела | Численность сотрудников в отделе | Навыки                  |
|---------------|--------------------------|-------------|-----------|-----------------|----------------------------------|-------------------------|
| 1             | Иванов Иван Иванович     | Программист | 3         | Отдел финансов  | 6                                | SQL, VBA, C++, Git, PHP |
| 2             | Петров Петр Петрович     | Курьер      | 7         | Отдел доставки  | 10                               |                         |
| 5             | Сергеев Сергей Сергеевич | Директор    |           |                 |                                  | SQL, VBA                |

1. Изменить название отдела финансов на «Финансовый департамент»
2. Добавить нового сотрудника в отдел доставки
3. Удалить из таблицы всех сотрудников отдела доставки в связи с переводом доставки на аутсорс

# Нормализация базы данных

Нормальная форма — требование, предъявляемое к структуре таблиц в теории реляционных баз данных для устранения из базы избыточных функциональных зависимостей между атрибутами (полями таблиц).

Метод нормальных форм (НФ) состоит в сборе информации об объектах решения задачи в рамках одного отношения и последующей декомпозиции этого отношения на несколько взаимосвязанных отношений на основе процедур нормализации отношений.

[Хорошая статья по НФ](#)

# Первая нормальная форма

Отношение находится в 1НФ, если все его атрибуты являются простыми, все используемые домены должны содержать только скалярные значения. Не должно быть повторений строк в таблице.

| ФИО Сотрудника - РК      | Выданное имущество              |
|--------------------------|---------------------------------|
| Иванов Иван Иванович     | Дырокол, степлер, ноутбук, мышь |
| Петров Петр Петрович     | Ноутбук, мышь                   |
| Сергеев Сергей Сергеевич | Ноутбук, мышь                   |



# Первая нормальная форма (корректная)

| ФИО Сотрудника           | Выданное имущество |
|--------------------------|--------------------|
| Иванов Иван Иванович     | Дырокол            |
| Иванов Иван Иванович     | Степлер            |
| Иванов Иван Иванович     | Ноутбук            |
| Иванов Иван Иванович     | Мышь               |
| Петров Петр Петрович     | Ноутбук            |
| Петров Петр Петрович     | Мышь               |
| Сергеев Сергей Сергеевич | Ноутбук            |
| Сергеев Сергей Сергеевич | Мышь               |

# Вторая нормальная форма

Отношение находится во 2НФ, если оно находится в 1НФ и каждый не ключевой атрибут неприводимо зависит от Первичного Ключа(ПК).  
Неприводимость означает, что в составе потенциального ключа отсутствует меньшее подмножество атрибутов, от которого можно также вывести данную функциональную зависимость.

| Марка автомобиля - РК | Модель автомобиля - РК | Цена автомобиля | Процент по кредиту |
|-----------------------|------------------------|-----------------|--------------------|
| Geely                 | Atlas                  | 35 200          | 1,9%               |
| Geely                 | Emgrand X7             | 32 000          | 1,9%               |
| Lada                  | Vesta                  | 19 610          | 14,99%             |

# Вторая нормальная форма (корректная)

| Марка автомобиля | Модель автомобиля | Цена автомобиля |
|------------------|-------------------|-----------------|
| Geely            | Atlas             | 35 200          |
| Geely            | Emgrand X7        | 32 000          |
| Lada             | Vesta             | 19 610          |

| Марка автомобиля | Процент по кредиту |
|------------------|--------------------|
| Geely            | 1,9%               |
| Lada             | 14,99%             |

# Третья нормальная форма

Отношение находится в 3НФ, когда находится во 2НФ и каждый не ключевой атрибут нетранзитивно зависит от первичного ключа.

Проще говоря, второе правило требует выносить все не ключевые поля, содержимое которых может относиться к нескольким записям таблицы в отдельные таблицы.

| Модель телефона - PK | Операционная система | Производитель ОС |
|----------------------|----------------------|------------------|
| iPhone 17 Pro Max    | iOS                  | Apple            |
| iPad Pro             | iOS                  | Apple            |
| Samsung S24          | Android              | Google           |

Модель -> ОС; ОС -> Производитель ОС;  
??? Модель -> Производитель ОС ???

# Третья нормальная форма (корректная)

| Модель телефона   | Операционная система |
|-------------------|----------------------|
| iPhone 17 Pro Max | iOS                  |
| iPad Pro          | iOS                  |
| Samsung S24       | Android              |

| Операционная система | Производитель ОС |
|----------------------|------------------|
| iOS                  | Apple            |
| Android              | Google           |

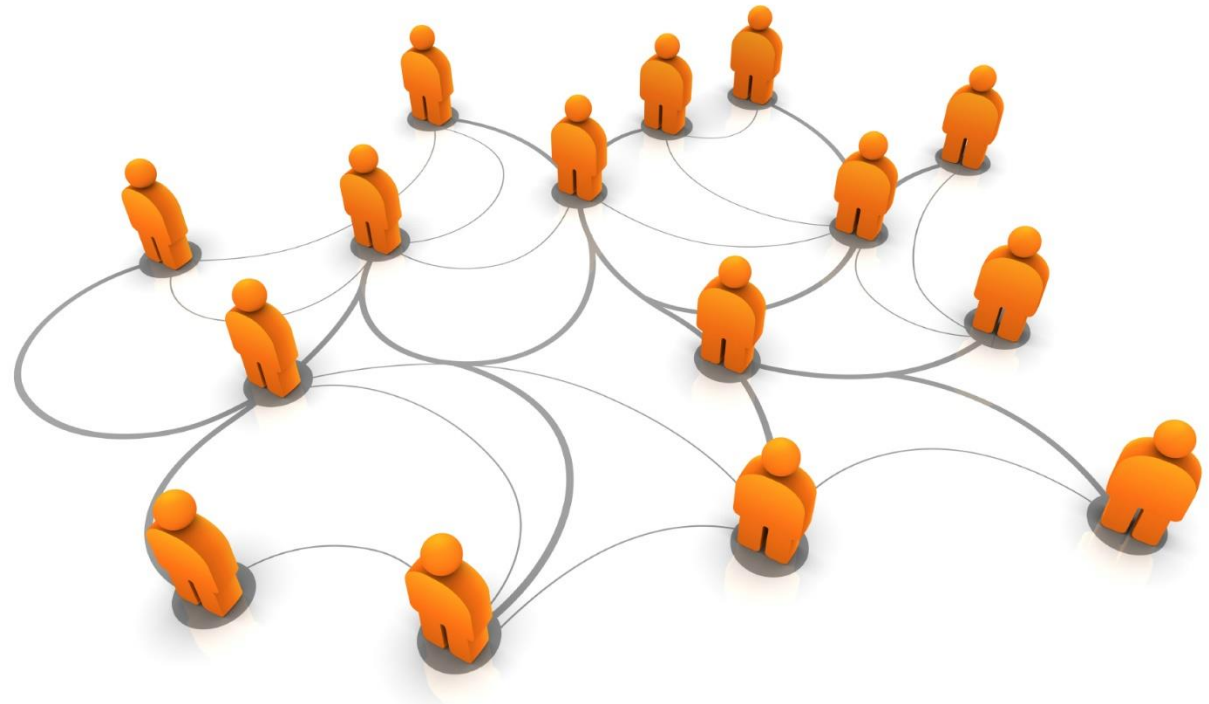
# Внешний ключ (Foreign key)

Внешний ключ - подмножество атрибутов некоторой переменной отношения R2, значения которых должны совпадать со значениями некоторого потенциального ключа некоторой переменной отношения R1.

- Внешний ключ не обязательно должен присутствовать в таблице
- Значения внешнего ключа могут быть не уникальными
- Значение внешнего ключа может быть NULL
- Внешний ключ может быть как частью первичного ключа, так и не ключевым атрибутом

# Связи между таблицами

- Один к одному (1:1)
- Один ко многим (1:N)
- Многие ко многим (N:M)



# Связь вида один к одному

| Гражданин                        | Паспорт        |
|----------------------------------|----------------|
| ID Гражданина                    | Номер паспорта |
| Имя                              | Дата выдачи    |
| Фамилия                          | Срок действия  |
| Отчество                         |                |
| Номер паспорта<br>(действующего) |                |



# Связь вида один ко многим

| Улица         |
|---------------|
| ID Улицы      |
| Название      |
| Протяженность |

| Дом               |
|-------------------|
| ID Дома           |
| Номер дома        |
| Количество этажей |
| Год постройки     |
| ID Улицы          |

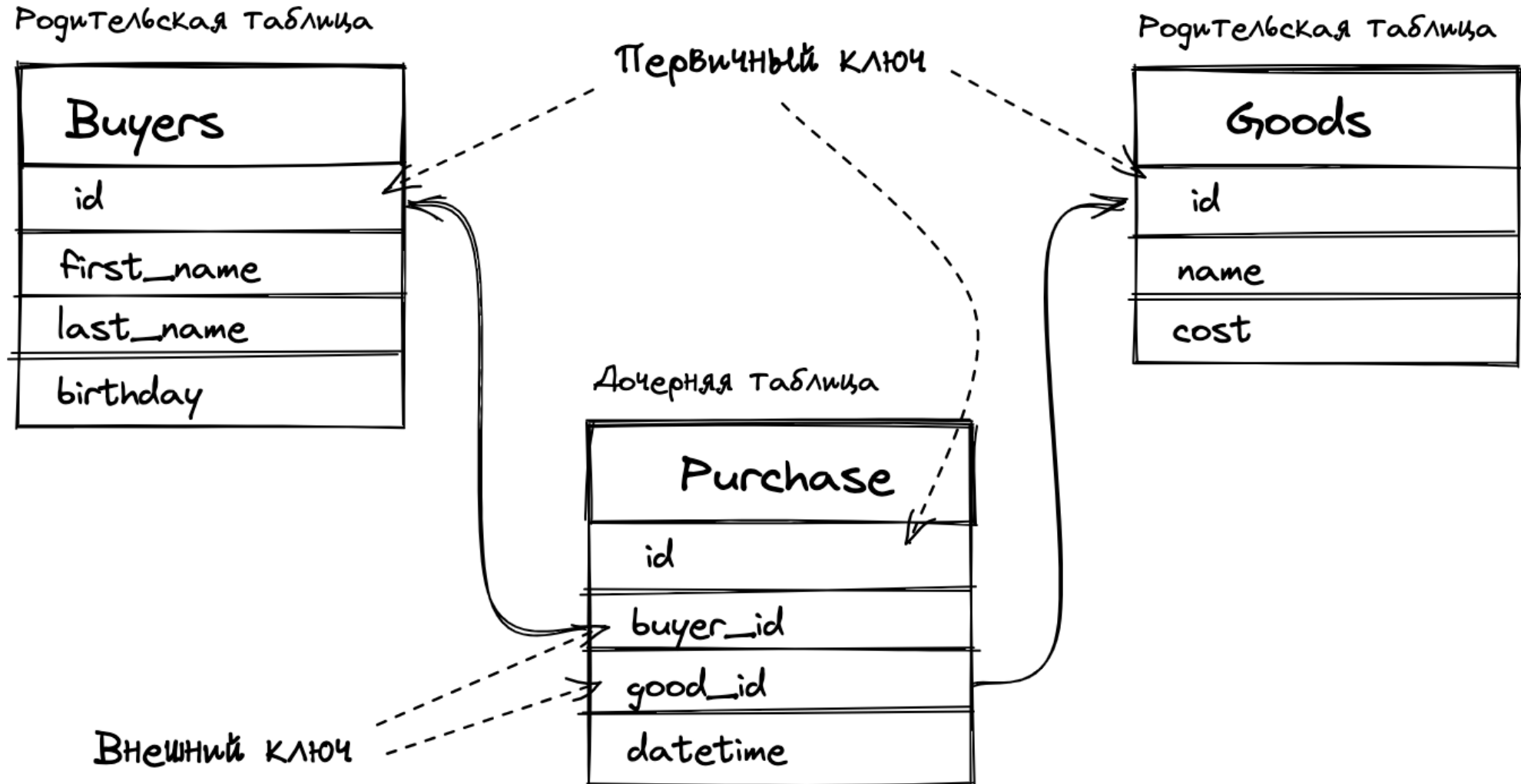
# Связь вида многие ко многим

| Комната              |
|----------------------|
| ID Комнаты           |
| Этаж                 |
| Уровень комфортности |
| Вместимость          |

| Бронирование      |
|-------------------|
| ID Бронирования   |
| Дата начала брони |
| Дата конца брони  |
| ID Комнаты        |
| ID Гостя          |

| Гость    |
|----------|
| ID Гостя |
| ФИО      |
| Телефон  |
| Email    |

# Многие ко многим и ключи



# NULL

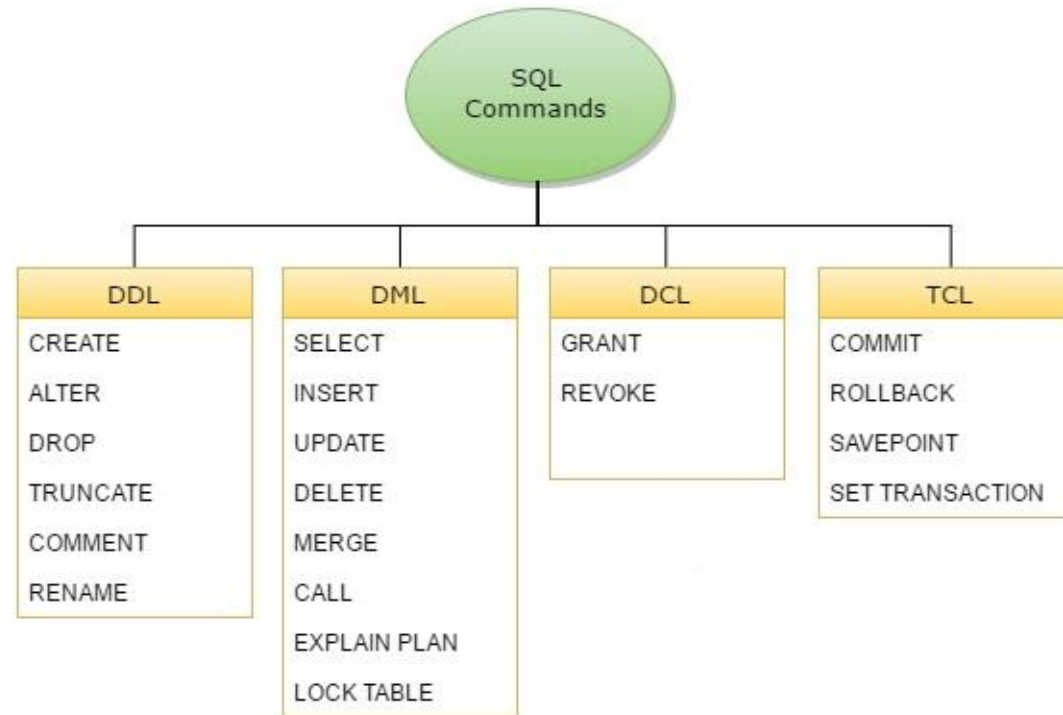
**SELECT 1 = NULL, 1 <> NULL, 1 < NULL, 1 > NULL;**

| 1 = NULL | 1 <> NULL | 1 < NULL | 1 > NULL |
|----------|-----------|----------|----------|
| null     | null      | null     | null     |

**SELECT 1 IS NULL, 1 IS NOT NULL, NULL IS NULL, NULL IS NOT NULL;**

| 1 IS NULL | 1 IS NOT NULL | NULL IS NULL | NULL IS NOT NULL |
|-----------|---------------|--------------|------------------|
| 0         | 1             | 1            | 0                |

# Команды языка SQL



# DDL

DDL - это короткое название языка определения данных, которое имеет дело с схемы и описания базы данных о том, как данные должны находиться в базы данных.

- CREATE - для создания базы данных и ее объектов типа (таблица, индекс, представления, процедура хранения, функция и триггеры)
- ALTER - изменяет структуру существующей базы данных
- DROP - удаление объектов из базы данных
- TRUNCATE - удалить все записи из таблицы, включая все пробелы, выделенные для записей, удалены
- COMMENT - добавление комментариев в словарь данных
- RENAME - переименовать объект

# DML

DML - это короткое имя языка манипулирования данными, который обрабатывает данные манипуляции и включает в себя большинство распространенных операторов SQL, таких как SELECT, INSERT, UPDATE, DELETE и т. Д., И он используется для хранения, изменения, извлечения, удалять и обновлять данные в базе данных.

- SELECT - получение данных из базы данных
- INSERT - вставить данные в таблицу
- UPDATE - обновляет существующие данные в таблице
- DELETE - удаление всех записей из таблицы базы данных
- Операция MERGE - UPSERT (вставка или обновление)
- CALL - вызов подпрограммы PL / SQL или Java
- ПЛАН EXPLAIN - интерпретация пути доступа к данным
- LOCK TABLE - контроль параллелизма

# DCL и TCL

DCL - это короткое имя языка управления данными, которое включает в себя команды таких как GRANT, и в основном касаются прав, разрешений и других управления системой базы данных.

- GRANT - позволяет пользователям получать доступ к базе данных
- REVOKE - вывести права доступа пользователей, заданные с помощью команды GRANT

TCL - это краткое название языка управления транзакциями, который имеет дело с транзакции в базе данных.

- COMMIT - совершает транзакцию
- ROLLBACK - откат транзакции в случае возникновения какой-либо ошибки
- SAVEPOINT - откат транзакционных точек внутри групп
- SET TRANSACTION - указать характеристики для транзакции



# CREATE

```
CREATE TABLE employees_demo
( employee_id    NUMBER(6)
, first_name     VARCHAR2(20)
, last_name      VARCHAR2(25)
  CONSTRAINT emp_last_name_nn_demo NOT NULL
, email          VARCHAR2(25)
  CONSTRAINT emp_email_nn_demo     NOT NULL
, phone_number   VARCHAR2(20)
, hire_date      DATE  DEFAULT SYSDATE
  CONSTRAINT emp_hire_date_nn_demo NOT NULL
, job_id         VARCHAR2(10)
  CONSTRAINT      emp_job_nn_demo  NOT NULL
, salary         NUMBER(8,2)
  CONSTRAINT      emp_salary_nn_demo NOT NULL
, commission_pct NUMBER(2,2)
, manager_id     NUMBER(6)
, department_id  NUMBER(4)
, dn             VARCHAR2(300)
, CONSTRAINT      emp_salary_min_demo
  CHECK (salary > 0)
, CONSTRAINT      emp_email_uk_demo
  UNIQUE (email)
) ;
```

```
CREATE TABLE countries_demo
( country_id     CHAR(2)
  CONSTRAINT country_id_nn_demo NOT NULL
, country_name   VARCHAR2(40)
, currency_name  VARCHAR2(25)
, currency_symbol VARCHAR2(3)
, region        VARCHAR2(15)
, CONSTRAINT      country_c_id_pk_demo
  PRIMARY KEY (country_id ) )
```

# Суррогатный ключ и AUTO\_INCREMENT

Table definition:

```
CREATE TABLE departments (  
  ID          NUMBER(10) NOT NULL,  
  DESCRIPTION VARCHAR2(50) NOT NULL);  
  
ALTER TABLE departments ADD (  
  CONSTRAINT dept_pk PRIMARY KEY (ID));  
  
CREATE SEQUENCE dept_seq START WITH 1;
```

Trigger definition:

```
CREATE OR REPLACE TRIGGER dept_bir  
BEFORE INSERT ON departments  
FOR EACH ROW  
  
BEGIN  
  SELECT dept_seq.NEXTVAL  
  INTO   :new.id  
  FROM   dual;  
END;  
/
```

# ALTER

```
ALTER TABLE table_name  
  ADD column_name column-definition;
```

```
ALTER TABLE table_name  
  MODIFY column_name column_type;
```

```
ALTER TABLE table_name  
  DROP COLUMN column_name;
```